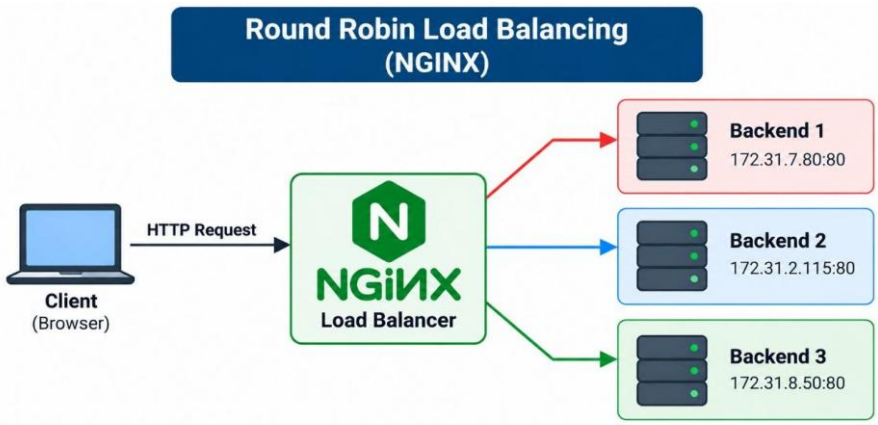
LOAD BALANCER ALGORITHEM – ROUND ROBIN USING NGINX



CREATE SECURITY GROUPS

1.create backend security group

1. ssh - 22 (my-ip)
2. http - 80 (loadbalancer-sg)

2.create loadbalancer security group

1. ssh - 22
2. http - 80

CREATE BACKEND SERVERS

1.create a backend server 1

1. name - backend-server-1
2. AMI - ubuntu
3. sg - backend-sg
4. user_data

```
sudo apt update
sudo apt install apache2 -y
echo '<h1>RESPONSE FROM BACKEND 1</h1>' \
| sudo tee /var/www/html/index.html
sudo systemctl enable --now apache2
```

2.create a backend server 2

1. name - backend-server-2
2. AMI - ubuntu
3. sg - backend-sg
4. user_data

```
sudo apt update
sudo apt install apache2 -y
echo '<h1>RESPONSE FROM BACKEND 2</h1>' \
| sudo tee /var/www/html/index.html
sudo systemctl enable --now apache2
```

CREATE LOAD-BALANCER SERVER

3.create a load balancer server

1. name - lb-server
2. ami - ubuntu
3. sg - loadbalancer-sg
4. launch instance

ACCESSING THE LOAD BALANCER

1. sudo apt update
2. sudo apt install nginx -y
3. curl http://BACKEND_1_PRIVATE_IP
4. curl http://BACKEND_2_PRIVATE_IP
5. sudo vim /etc/nginx/sites-available/default

```
upstream backend_servers {
   server BACKEND_1_PRIVATE_IP:80;
   server BACKEND_2_PRIVATE_IP:80;
}

server {
   listen 80 default_server;
   server_name _;


   location / {
      proxy_pass http://backend_servers;


      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
   }
}
```

6. sudo nginx -t
7. sudo systemctl enable --now nginx
8. sudo systemctl reload nginx

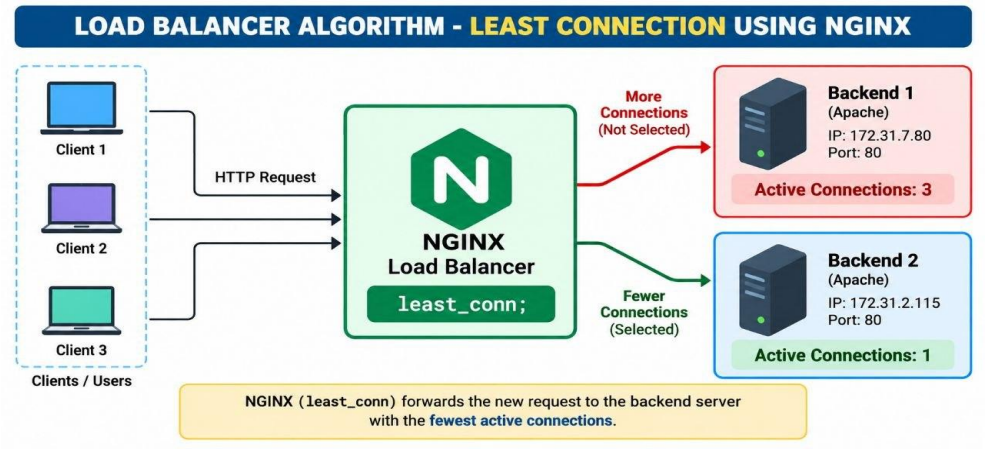
TEST THE LOAD BALANCER

1. http://Your_Load_balancer_ip

YOU GET THE RESPONSE FROM

1. backend-1
2. backend-2

LOAD BALANCER ALGORITHEM - LEAST CONNECTION USING NGINX



CREATE SECURITY GROUPS

1.create backend security group

3. ssh - 22 (my-ip)
4. http - 80 (loadbalancer-sg)

2.create loadbalancer security group

3. ssh - 22
4. http - 80

CREATE BACKEND SERVERS

1.create a backend server 1

5. name - backend-server-1
6. AMI - ubuntu
7. sg - backend-sg
8. user_data

```
sudo apt update
sudo apt install apache2 -y
echo '<h1>RESPONSE FROM BACKEND 1</h1>' \
| sudo tee /var/www/html/index.html
sudo systemctl enable --now apache2
```

2.create a backend server 2

5. name - backend-server-2
6. AMI - ubuntu
7. sg - backend-sg
8. user_data

```
sudo apt update
sudo apt install apache2 -y
echo '<h1>RESPONSE FROM BACKEND 2</h1>' \
| sudo tee /var/www/html/index.html
sudo systemctl enable --now apache2
```

CREATE LOAD-BALANCER SERVER

3.create a load balancer server

1. name - lb-server
2. ami - ubuntu
3. sg - loadbalancer-sg
4. launch instance

ACCESSING THE LOAD BALANCER

1. sudo apt update
2. sudo apt install nginx -y
3. curl http://BACKEND_1_PRIVATE_IP
4. curl http://BACKEND_2_PRIVATE_IP
5. sudo vim /etc/nginx/sites-available/default

```
upstream backend_servers {
   least_conn;

   server BACKEND_1_PRIVATE_IP:80;
   server BACKEND_2_PRIVATE_IP:80;
}

server {
   listen 80 default_server;
   server_name _;


   location / {
      proxy_pass http://backend_servers;


      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
   }
}
```

6. sudo nginx -t
7. sudo systemctl enable --now nginx
8. sudo systemctl reload nginx

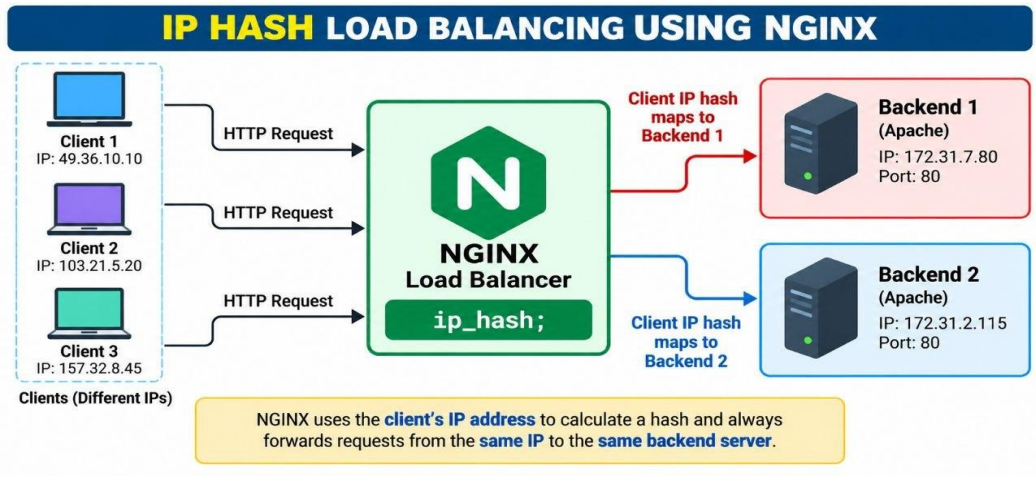
TEST THE LOAD BALANCER

2. http://Your_Load_balancer_ip

YOU GET THE RESPONSE FROM

3. backend-1
4. backend-2

LOAD BALANCER ALGORITHEM – IP HASH



CREATE SECURITY GROUPS

1.create backend security group

5. ssh - 22 (my-ip)
6. http - 80 (loadbalancer-sg)

2.create loadbalancer security group

5. ssh - 22
6. http - 80

CREATE BACKEND SERVERS

1.create a backend server 1

9. name - backend-server-1
10. AMI - ubuntu
11. sg - backend-sg
12. user_data

```
sudo apt update
sudo apt install apache2 -y
echo '<h1>RESPONSE FROM BACKEND 1</h1>' \
| sudo tee /var/www/html/index.html
sudo systemctl enable --now apache2
```

2.create a backend server 2

9. name - backend-server-2
10. AMI - ubuntu
11. sg - backend-sg
12. user_data

```
sudo apt update
sudo apt install apache2 -y
echo '<h1>RESPONSE FROM BACKEND 2</h1>' \
| sudo tee /var/www/html/index.html
```

sudo systemctl enable --now apache2

CREATE LOAD-BALANCER SERVER

3.create a load balancer server

5. 1.name - lb-server
6. 2.ami - ubuntu
7. 3.sg - loadbalancer-sg
8. 4.launch instance

ACCESSING THE LOAD BALANCER

9. sudo apt update
10. sudo apt install nginx -y
11. curl http://BACKEND_1_PRIVATE_IP
12. curl http://BACKEND_2_PRIVATE_IP
13. sudo vim /etc/nginx/sites-available/default

```
upstream backend_servers {
   ip_hash;

   server BACKEND_1_PRIVATE_IP:80;
   server BACKEND_2_PRIVATE_IP:80;
}

server {
   listen 80 default_server;
   server_name _;


   location / {
      proxy_pass http://backend_servers;


      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
   }
}
```

14. sudo nginx -t
15. sudo systemctl enable --now nginx
16. sudo systemctl reload nginx

TEST THE LOAD BALANCER

3. http://Your_Load_balancer_ip

YOU SHOULD SEE EITHER…

1. RESPONSE FROM BACKEND 1

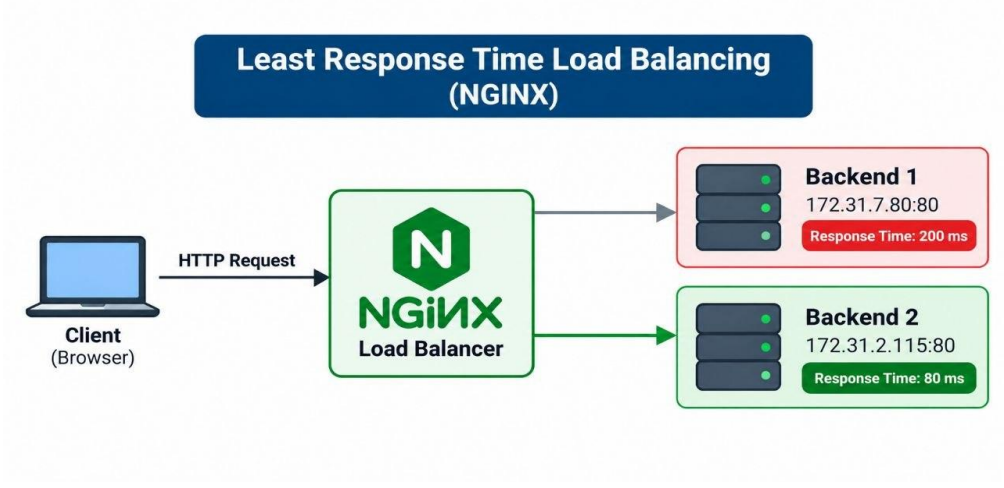
2. RESPONSE FROM BACKEND 2

EXPECTED RESULT:

1. Requests from the same client IP should generally continue reaching the same backend, provided the backend configuration remains unchanged.
2. The new network may produce a different client IP and therefore select a different backend.
3. However, it can still select the same backend; IP Hash does not guarantee that different clients will reach different servers

---

LOAD BALANCER ALGORITHEM – LEAST-RESPONSE TIME



CREATE SECURITY GROUPS

1.create backend security group

1. ssh - 22 (my-ip)
2. http - 80 (loadbalancer-sg)

2.create loadbalancer security group

1. ssh - 22
2. http - 80

CREATE BACKEND SERVERS

1.create a backend server 1

1. name - backend-server-1
2. AMI - ubuntu
3. sg - backend-sg
4. user_data

```
sudo apt update
sudo apt install apache2 -y
echo '<h1>RESPONSE FROM BACKEND 1</h1>' \
| sudo tee /var/www/html/index.html
sudo systemctl enable --now apache2
```

2.create a backend server 2

1. name - backend-server-2
2. AMI - ubuntu
3. sg - backend-sg
4. user_data
   sudo apt update
   sudo apt install apache2 -y
   echo '<h1>RESPONSE FROM BACKEND 2</h1>' \
   | sudo tee /var/www/html/index.html
   sudo systemctl enable --now apache2

CREATE LOAD-BALANCER SERVER

create a load balancer server

1. name - lb-server
2. ami - ubuntu
3. sg - loadbalancer-sg
4. launch instance

INSTALLATION OF NGINX ON LOAD-BALANCER SERVER

1. sudo apt update
2. sudo apt install -y curl gnupg2 ca-certificates lsb-release ubuntu-keyring
3. curl -fsSL https://nginx.org/keys/nginx_signing.key \
   | sudo gpg --dearmor --yes -o /usr/share/keyrings/nginx-archive-keyring.gpg
4. gpg --show-keys /usr/share/keyrings/nginx-archive-keyring.gpg
5. echo "deb [signed-by=/usr/share/keyrings/nginx-archive-keyring.gpg]
   https://nginx.org/packages/mainline/ubuntu resolute nginx" \
   | sudo tee /etc/apt/sources.list.d/nginx.list
6. apt update
7. apt policy nginx
8. sudo apt install nginx=1.31.8-1~resolute
9. nginx -v
10. curl http://BACKEND_1_PRIVATE_IP
11. curl http://BACKEND_2_PRIVATE_IP
12. vim /etc/nginx/conf.d/default.conf

```
upstream backend_servers {
   least_time header;

   server 172.31.7.80:80;
   server 172.31.2.115:80;
}

server {
   listen 80 default_server;
   server_name _;

   location / {
```

```
        proxy_pass http://backend_servers;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

13. nginx -t
14. systemctl enable nginx
15. systemctl reload nginx
16. systemctl status nginx --pager

TEST THE LOAD BALANCER

1. http://Your_Load_balancer_ip

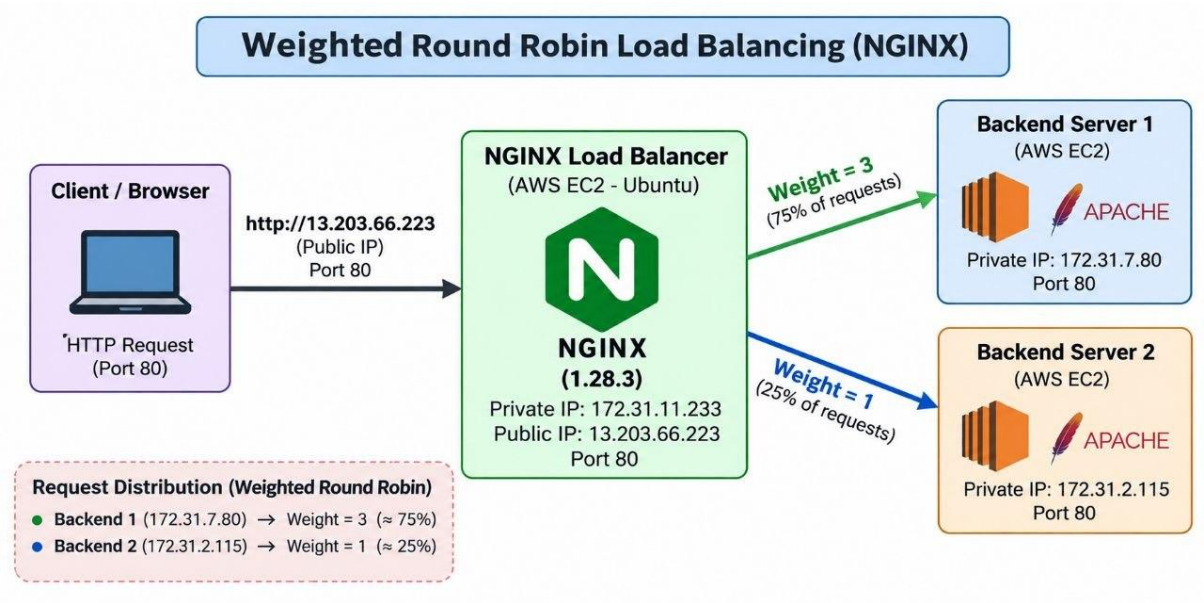
YOU SHOULD SEE EITHER…

3. RESPONSE FROM BACKEND 1
4. RESPONSE FROM BACKEND 2

EXPLANATION:

1. Least Response Time Load Balancing is an algorithm that routes incoming requests to the backend server with the lowest average response time.
2. NGINX considers the time taken to receive response headers and the number of active connections to help select a server.
3. It helps improve application responsiveness by directing traffic toward faster-performing backend servers.

LOAD BALANCER ALGORITHEM – WEIGHTED ROUND ROBIN



CREATE SECURITY GROUPS

1.create backend security group

1. ssh - 22 (my-ip)
2. http - 80 (loadbalancer-sg)

2.create loadbalancer security group

1. ssh - 22
2. http - 80

CREATE BACKEND SERVERS

1.create a backend server 1

1. name - backend-server-1
2. AMI - ubuntu
3. sg - backend-sg
4. user_data

```
sudo apt update
sudo apt install apache2 -y
echo '<h1>RESPONSE FROM BACKEND 1</h1>' \
| sudo tee /var/www/html/index.html
sudo systemctl enable --now apache2
```

2.create a backend server 2

1. name - backend-server-2
2. AMI - ubuntu
3. sg - backend-sg
4. user_data

```
sudo apt update
sudo apt install apache2 -y
echo '<h1>RESPONSE FROM BACKEND 2</h1>' \
| sudo tee /var/www/html/index.html
sudo systemctl enable --now apache2
```

CREATE LOAD-BALANCER SERVER

3.create a load balancer server

1. name - lb-server
2. ami - ubuntu
3. sg - loadbalancer-sg
4. launch instance

ACCESSING THE LOAD BALANCER

1. sudo apt update
2. sudo apt install nginx -y
3. curl http://BACKEND_1_PRIVATE_IP
4. curl http://BACKEND_2_PRIVATE_IP

5. sudo vim /etc/nginx/sites-available/default

```
upstream backend_servers {
   server 172.31.7.80:80 weight=3;
   server 172.31.2.115:80 weight=1;
}

server {
   listen 80 default_server;
   listen [::]:80 default_server;

   server_name _;

   location / {
      proxy_pass http://backend_servers;

      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
   }
}
```

6. sudo nginx -t
7. sudo systemctl enable --now nginx
8. sudo systemctl reload nginx

TEST THE LOAD BALANCER

1. http://Your_Load_balancer_ip

YOU GET THE RESPONSE FROM

1. backend-1
2. backend-2

IMPORTANT:

1. The weights 3 and 1 mean Backend 1 receives approximately three times as many requests as Backend 2 over time. The distribution of a small number of requests may vary.
2. Weighted Round Robin is useful when backend servers have different processing capacities.
3. A server with a higher weight receives a larger share of requests, helping distribute traffic according to the capacity assigned to each backend.
